

Air University

Department of Computer Science

Introduction to Parallel & Distributed Computing

Lab Manual

Lab 04: Concurrency Issues

Item	Details
Course	Introduction to Parallel & Distributed Computing
Course Code	CS426L
Duration	3 Hours
Instructor	Muhammad Nauman Zafar Hashmi (Lecturer, CS Dept.)
Semester	VII C (FALL26)

1. Lab Overview

Item	Details
Lab Number	04
Lab Title	Concurrency Issues
Duration	3 Hours
Programming Language	C (GCC, POSIX Threads / pthreads)
Development Environment	WSL (Ubuntu) with GCC and ThreadSanitizer, accessed from Windows Terminal or the VS Code integrated terminal (same WSL setup as Lab 02)
Main Concepts	Shared-state hazard, race condition, lost update, critical section, mutual exclusion, atomic read-modify-write operations, synchronization overhead
Main C Features / Tools	pthread_create(), pthread_join(), pthread_mutex_lock() / pthread_mutex_unlock(), __sync_fetch_and_add(), C11 <stdatomic.h> (atomic_fetch_add), -fsanitize=thread (ThreadSanitizer), clock_gettime()

After completing this laboratory, students should be able to:

1. implement a multi-threaded counter with an unsynchronized shared-state update, run it repeatedly, and record the inconsistent results caused by the race condition;
2. recompile a program with ThreadSanitizer (`-fsanitize=thread`) and interpret the data-race report it produces;

3. fix the race by wrapping the shared-state update in a critical section protected by a mutex, and re-verify correctness;
4. reimplement the fix with an atomic increment (GCC `__sync_fetch_and_add` or a C11 atomic) instead of a mutex, and compare its correctness and performance against the critical-section version.

Students should be familiar with basic C syntax, compiling with `gcc` inside WSL (Lab 02), and the thread creation and joining concepts from Lab 02 and Lab 03 (`Start()/Join()` in C# correspond to `pthread_create()/pthread_join()` in C).

Note on language choice. Lab 03 used C# and `System.Threading`. This lab returns to C because ThreadSanitizer (`-fsanitize=thread`) and GCC's `__sync` atomic builtins are features of the GCC toolchain. The concepts carry over directly: the C# equivalents of the fixes in this lab are the `lock` statement (or `Monitor`) for the mutex, and `Interlocked.Increment` for the atomic increment.

2. Setting Up the Development Environment

This laboratory reuses the WSL (Ubuntu) environment configured in Lab 02. All commands in this lab are run inside the WSL terminal, not in plain PowerShell.

2.1 Confirm the Toolchain

Open a WSL terminal and confirm that GCC is available:

```
gcc --version
nproc
```

`nproc` reports the number of logical cores; note this value, since it affects how strongly the threads contend for the shared counter. If `gcc` is not recognized, install the build tools:

```
sudo apt update
sudo apt install build-essential
```

Verify that ThreadSanitizer links correctly with this one-line test:

```
echo 'int main(void){return 0;}' > tsan_test.c
gcc -fsanitize=thread tsan_test.c -o tsan_test && ./tsan_test && echo "TSan OK"
```

If this fails with a message such as "cannot find -ltsan", install the ThreadSanitizer runtime library for your GCC version. If the program starts but aborts with "unexpected memory mapping", see the Troubleshooting Guide (Section 10).

2.2 Create a Working Folder for This Lab

1. In the WSL terminal, create and enter a folder for this lab: `mkdir -p ~/PDC_Lab04 && cd ~/PDC_Lab04`.
2. Optionally open it in VS Code with `code .` (requires the VS Code WSL extension from Lab 02).

Each task below uses its own C source file inside this folder: `race.c`, `race_mutex.c`, and `counter_bench.c`.

3. Background: Races, Mutexes, and Atomics

Review the following terminology before starting the tasks.

- **Shared state:** data that more than one thread can read and write, such as a global variable or heap object. Threads in one process share the same address space (Lab 03), so shared state is easy to create, and easy to corrupt.
- **Race condition / data race:** a defect in which the result depends on the timing of two or more threads accessing the same memory location, where at least one access is a write and the accesses are not ordered by any synchronization.
- **Lost update:** the typical symptom of a race on a counter. The statement `counter++` looks like one operation but is really three machine-level steps: load the value, add one, store the result. Two threads can interleave these steps so that one increment overwrites the other, as shown below.

Step	Thread A	Thread B	counter in memory
1	load counter (reads 5)		5
2		load counter (reads 5)	5
3	add 1, store 6		6
4		add 1, store 6	6 (should be 7: one update lost)

- **Critical section:** a region of code that accesses shared state and must be executed by at most one thread at a time.
- **Mutex (mutual-exclusion lock):** a synchronization object that enforces this rule. A thread calls `pthread_mutex_lock()` before entering the critical section and `pthread_mutex_unlock()` on leaving it; other threads that try to lock in the meantime are made to wait.
- **Atomic operation:** a read-modify-write that the hardware performs as one indivisible step, so no other thread can observe or interleave with the middle of it. GCC exposes this through `__sync_fetch_and_add()`; C11 provides `atomic_fetch_add()` in `<stdatomic.h>`. An atomic increment needs no lock, but it only protects a single variable.
- **ThreadSanitizer (TSan):** a dynamic race detector built into GCC and Clang. Compiling with `-fsanitize=thread` instruments every memory access; at run time TSan reports pairs of conflicting accesses that are not ordered by synchronization. It detects the race itself, even in runs where the printed result happens to look correct, but it slows the program down considerably (often 5x to 15x), so it is a debugging tool, not a benchmarking configuration.
- **Contention:** the slowdown that occurs when many threads repeatedly compete for the same lock or the same memory location. Both the mutex and the atomic version in Task 4 are subject to it, to different degrees.

4. Task 1: Unsynchronized Counter (Shared-State Hazard)

Implement a multi-threaded counter increment with no synchronization; run it 10 times and record the inconsistent results.

4.1 Approach

Several threads each increment one global counter a fixed number of times. If the increments were performed correctly, the final value would equal the number of threads multiplied by the number of increments per thread. Because `counter++` is not atomic, updates are lost, and the final value is usually smaller than expected and different from run to run.

4.2 Required Steps

1. Create a file named `race.c` in `~/PDC_Lab04`.
2. Declare a global `long counter = 0;` shared by all threads.
3. Write a worker function that executes `counter++` a fixed number of times (`INCREMENTS_PER_THREAD`, 1,000,000 by default) with no lock and no atomic operation.
4. In `main()`, create `NUM_THREADS` (4) threads with `pthread_create()`, join all of them with `pthread_join()`, then print the expected value and the actual value of `counter`.
5. Compile without optimization so that the increment is not transformed by the compiler:
`gcc -O0 -pthread race.c -o race.`
6. Run the program **10 times** (for example, for `i` in `$(seq 1 10); do ./race;` done) and record the results in the table in Section 4.4.

4.3 Suggested Program Skeleton (race.c)

```
#include <stdio.h>
#include <pthread.h>

#define NUM_THREADS 4
#ifndef INCREMENTS_PER_THREAD
#define INCREMENTS_PER_THREAD 1000000
#endif

long counter = 0;           /* shared state */

void *worker(void *arg)
{
    for (int i = 0; i < INCREMENTS_PER_THREAD; i++) {
        counter++;          /* unsynchronized read-modify-write */
    }
    return NULL;
}

int main(void)
{
    pthread_t threads[NUM_THREADS];

    for (int i = 0; i < NUM_THREADS; i++)
        pthread_create(&threads[i], NULL, worker, NULL);
    for (int i = 0; i < NUM_THREADS; i++)
        pthread_join(threads[i], NULL);

    printf("Expected: %ld\n", (long)NUM_THREADS * INCREMENTS_PER_THREAD);
    printf("Actual:    %ld\n", counter);
    return 0;
}
```

Note: `INCREMENTS_PER_THREAD` is wrapped in `#ifndef` so that it can be overridden from the command line with `-DINCREMENTS_PER_THREAD=100000` in Task 2.

4.4 Record Your Results

Run	Expected Value	Actual Value	Lost Updates (Expected – Actual)
1			
2			
3			
4			
5			
6			
7			
8			
9			
10			

Number of logical cores on your machine (nproc): _____

4.5 Discussion Questions

1. Did the actual value ever equal the expected value? Were any two runs identical? Why is the result non-deterministic even though the source code never changes?
2. Using the load / add / store breakdown of `counter++` from Section 3, explain how one increment can be lost when two threads interleave.
3. Suppose `INCREMENTS_PER_THREAD` were reduced to 100. Would you expect the race to disappear, or merely to become harder to observe? Explain why "the output looked correct" is not evidence that a program is free of races.

5. Task 2: Detecting the Race with ThreadSanitizer

Recompile the Task 1 program with ThreadSanitizer (`-fsanitize=thread`) and interpret the race-condition report.

5.1 Required Steps

1. Compile the **unchanged** `race.c` with ThreadSanitizer, debug symbols (`-g`, so the report shows line numbers), and a smaller workload, since TSan makes the program much slower: `gcc -fsanitize=thread -g -O1 -pthread -DINCREMENTS_PER_THREAD=100000 race.c -o race_tsan.`
2. Run `./race_tsan`. TSan writes its report to standard error; capture it with `./race_tsan 2> tsan_report.txt` so it can be included in your submission.
3. Locate the following parts of the first warning in the report: the header line `WARNING: ThreadSanitizer: data race`; the two conflicting accesses (the current access and

the "Previous" access), each with its thread ID and source line; the "Location is global ..." line naming the variable; the lines showing where each thread was created; and the closing SUMMARY line.

4. Run the TSan build at least three times. Note whether the report and the final counter value change between runs.
5. Fill in the interpretation table in Section 5.3 using your own report.

5.2 What a Report Looks Like

The abbreviated report below shows the general shape only. Addresses, thread numbers, and line numbers on your machine will differ.

```
=====
WARNING: ThreadSanitizer: data race (pid=1234)
  Write of size 8 at 0x... by thread T2:
    #0 worker race.c:13 (race_tsan+0x...)

  Previous write of size 8 at 0x... by thread T1:
    #0 worker race.c:13 (race_tsan+0x...)

  Location is global 'counter' of size 8 at 0x... (race_tsan+0x...)

  Thread T2 (tid=..., running) created by main thread at:
    #0 pthread_create ...
    #1 main race.c:23 (race_tsan+0x...)

SUMMARY: ThreadSanitizer: data race race.c:13 in worker
=====
```

5.3 Record Your Interpretation

Item	Your Observation
Type of access #1 (read/write) and source line	
Type of access #2 ("Previous" access) and source line	
Memory location reported (variable name)	
Threads involved (IDs) and where they were created	
Number of warnings printed in one run	
Final counter value printed by the TSan build	

5.4 Discussion Questions

1. Which line of `race.c` did TSan blame for the race? Why are both conflicting accesses reported on the same line?
2. TSan may report a race even in a run where the final counter value was exactly right. How can a dynamic detector flag a race that did not visibly corrupt the result?

- Task 1 showed the symptom of the race (a wrong total); TSan shows its cause. Explain the difference and why a tool like TSan is needed in addition to testing the output.
- Why should the TSan build not be used for the timing measurements in Task 4?

6. Task 3: Fixing the Race with a Mutex (Critical Section)

Fix the race by wrapping the shared-state update in a critical section protected by a mutex, and re-verify correctness.

6.1 Required Steps

- Copy `race.c` to `race_mutex.c`: `cp race.c race_mutex.c`.
- Declare a global mutex: `pthread_mutex_t counter_lock = PTHREAD_MUTEX_INITIALIZER;`
- In the worker, surround **only** the `counter++` statement with `pthread_mutex_lock(&counter_lock)` and `pthread_mutex_unlock(&counter_lock)`. Do not place the lock around the whole loop.
- Compile with `gcc -O0 -pthread race_mutex.c -o race_mutex` and run it **10 times**. Record the results in the table in Section 6.3.
- Recompile the fixed program with ThreadSanitizer (`gcc -fsanitize=thread -g -O1 -pthread -DINCREMENTS_PER_THREAD=100000 race_mutex.c -o race_mutex_tsan`), run it, and confirm that **no** data-race warning is printed.

6.2 Suggested Program Skeleton (`race_mutex.c`, changes only)

```
long counter = 0;
pthread_mutex_t counter_lock = PTHREAD_MUTEX_INITIALIZER;

void *worker(void *arg)
{
    for (int i = 0; i < INCREMENTS_PER_THREAD; i++) {
        /* Your task: lock, increment counter, unlock */
    }
    return NULL;
}
```

6.3 Record Your Results

Run	Expected Value	Actual Value	Correct? (Y/N)	TSan Warnings (count)
1				n/a
2				n/a
3				n/a
4				n/a
5				n/a
6				n/a

Run	Expected Value	Actual Value	Correct? (Y/N)	TSan Warnings (count)
7				n/a
8				n/a
9				n/a
10				n/a
TSan build				

6.4 Discussion Questions

1. Was the actual value equal to the expected value in all 10 runs? Why is this on its own not conclusive proof that the race is fixed, and what additional evidence did the TSan run provide?
2. `main()` reads `counter` after the joins without taking the lock. Why is that safe here? (Hint: what does `pthread_join()` guarantee about the finished threads' writes?)
3. What would happen to correctness if only three of the four threads used the lock and one still executed a bare `counter++`?
4. The lock is placed around each single increment rather than around the whole loop. What is the trade-off between the two placements in terms of correctness, parallelism, and lock overhead?

7. Task 4: Atomic Increment vs. Critical Section

Reimplement the fix using an atomic increment (GCC `__sync_fetch_and_add` or a C11 atomic) instead of a mutex, and compare correctness and performance against the critical-section version.

7.1 Required Steps

1. Create `counter_bench.c`, a single program in which the increment method is selected at compile time: `-DUSE_MUTEX` (Task 3's critical section), `-DUSE_SYNC` (GCC `__sync_fetch_and_add`), or `-DUSE_C11` (C11 `atomic_fetch_add`). Implementing at least one atomic variant is required; the skeleton supports both.
2. Time only the thread creation, work, and joining using `clock_gettime(CLOCK_MONOTONIC, ...)`, and print the version name, expected value, actual value, whether they match, and the elapsed time in milliseconds.
3. Build all versions with optimization enabled (and **without** TSan) for timing: `gcc -O2 -pthread -std=gnu11 -DUSE_MUTEX counter_bench.c -o bench_mutex`, and likewise `-DUSE_SYNC` $\rightarrow$ `bench_sync`, `-DUSE_C11` $\rightarrow$ `bench_c11`.
4. Run each executable **5 times** and record every elapsed time and the correctness result in the table in Section 7.3. Compute the average time of each version.
5. Verify the atomic versions with TSan on a reduced workload, for example `gcc -fsanitize=thread -g -O1 -pthread -std=gnu11 -DUSE_SYNC -`

DINCREMENTS_PER_THREAD=100000 counter_bench.c -o bench_sync_tsan, and confirm that no data-race warning is printed.

6. Compute the ratio (average mutex time / average atomic time) and record it.

7.2 Suggested Program Skeleton (counter_bench.c)

```
#include <stdio.h>
#include <pthread.h>
#include <stdatomic.h>
#include <time.h>

#ifndef NUM_THREADS
#define NUM_THREADS 4
#endif
#ifndef INCREMENTS_PER_THREAD
#define INCREMENTS_PER_THREAD 1000000
#endif

#if defined(USE_MUTEX)
    static long counter = 0;
    static pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
    #define INCREMENT() do { pthread_mutex_lock(&lock); counter++; \
                           pthread_mutex_unlock(&lock); } while (0)
    #define READ_COUNTER() counter
    #define VERSION "mutex"
#elif defined(USE_SYNC)
    static long counter = 0;
    #define INCREMENT() __sync_fetch_and_add(&counter, 1)
    #define READ_COUNTER() counter
    #define VERSION "__sync_fetch_and_add"
#elif defined(USE_C11)
    static atomic_long counter = 0;
    #define INCREMENT() atomic_fetch_add(&counter, 1)
    #define READ_COUNTER() atomic_load(&counter)
    #define VERSION "C11 atomic"
#else
    #error "Compile with -DUSE_MUTEX, -DUSE_SYNC or -DUSE_C11"
#endif

static double now_ms(void)
{
    struct timespec ts;
    clock_gettime(CLOCK_MONOTONIC, &ts);
    return ts.tv_sec * 1000.0 + ts.tv_nsec / 1e6;
}

static void *worker(void *arg)
{
    for (int i = 0; i < INCREMENTS_PER_THREAD; i++)
        INCREMENT();
    return NULL;
}

int main(void)
{
    pthread_t threads[NUM_THREADS];
    double start = now_ms();

    /* Your task: create NUM_THREADS threads running worker(), then join
       every one of them. */

    double elapsed = now_ms() - start;
    long expected = (long)NUM_THREADS * INCREMENTS_PER_THREAD;
    long actual = READ_COUNTER();
    printf("[%s] expected=%ld actual=%ld correct=%s time=%.2f ms\n",
           VERSION, expected, actual,
           expected == actual ? "YES" : "NO", elapsed);
}
```

```
    return 0;
}
```

7.3 Record Your Results

Run	Mutex Time (ms)	__sync Atomic Time (ms)	C11 Atomic Time (ms) (optional)
1			
2			
3			
4			
5			
Average			
Correct in all runs? (Y/N)			

Ratio (average mutex time / average atomic time): _____

Number of threads and increments per thread used: _____

7.4 Discussion Questions

1. Did the atomic version(s) produce the correct total in every run? Compare this with the unsynchronized program in Task 1.
2. Which version was faster on your machine, and by roughly what factor? Give two reasons why the atomic increment tends to be cheaper than a lock/unlock pair around the same increment.
3. Even the atomic version is far slower than a single thread incrementing an unshared variable. What causes this, given that all threads write the same memory location?
4. An atomic increment protects one variable. Give an example of an update that an atomic operation cannot make safe but a mutex can (for instance, an update that must change two related variables together), and explain why.
5. How would you expect the ratio of mutex time to atomic time to change with 1 thread, and with more threads than cores? Justify your answer, and if time permits, test it by recompiling with `-DNUM_THREADS=1` and `-DNUM_THREADS=8`.

8. Experimental Practice

For meaningful and reproducible results across this laboratory:

1. close other CPU-intensive applications before running Task 4, since background load affects the timings and the amount of contention;
2. use `-O0` for Tasks 1 and 3 so the race and its fix are easy to observe, and `-O2` (Release-style) for the timing runs in Task 4; use the same compiler flags, thread count, and increment count for every version being compared;

3. never record timings from a ThreadSanitizer build; use TSan builds only to check for race warnings, on a reduced workload;
4. run each program the number of times the task specifies (10 for Tasks 1 and 3, 5 per version for Task 4) and confirm that the pattern is consistent between runs;
5. keep the raw terminal output (and `tsan_report.txt`) from each task alongside your completed tables so recorded values can be verified.

9. Expected Findings

By completing this laboratory, students should observe that:

- the unsynchronized counter almost never reaches the expected total, and its final value differs from run to run, because `counter++` is a non-atomic load/add/store sequence and concurrent increments are lost;
- ThreadSanitizer reports a data race on the shared `counter`, identifying the conflicting accesses, their source line, and the threads involved, and can do so even when the output happens to be correct;
- wrapping the increment in a mutex-protected critical section makes the result correct on every run, and the ThreadSanitizer report for the fixed program is clean;
- an atomic increment (`__sync_fetch_and_add` or C11 `atomic_fetch_add`) is equally correct and, in this heavily contended workload, is typically faster than the mutex version, though results depend on the machine and thread count and both remain slower than uncontended, unshared work;
- atomics are suited to protecting a single variable, while a mutex is needed when several pieces of shared state must be updated together as one critical section.

10. Troubleshooting Guide

Problem	Likely Cause	Action
undefined reference to <code>pthread_create</code> at link time	The <code>-pthread</code> flag was omitted	Add <code>-pthread</code> to the gcc command line (on both compile and link).
Task 1 prints the expected value every time	Too few increments, too few threads, or an optimized build hiding the interleaving	Compile with <code>-O0</code> , keep 4 or more threads, and increase <code>INCREMENTS_PER_THREAD</code> (for example to 10,000,000).
TSan aborts with "FATAL: ThreadSanitizer: unexpected memory mapping"	Address-space randomization settings in newer Linux kernels (common on WSL2) are incompatible with the TSan runtime	Run <code>sudo sysctl vm.mmap_rnd_bits=28</code> in WSL, then run the TSan build again.
TSan prints no warning for the racy program	The program was not compiled with <code>-fsanitize=thread</code> , or the old non-TSan executable was run	Recompile with the flag, use a distinct output name such as <code>race_tsan</code> , and run that file.

Problem	Likely Cause	Action
TSan report shows no line numbers	The program was compiled without <code>-g</code>	Add <code>-g</code> to the compile command.
TSan build runs for a very long time	TSan slows programs considerably and the full workload was used	Reduce the workload with <code>-DINCREMENTS_PER_THREAD=100000</code> .
Mutex version hangs	A <code>pthread_mutex_unlock()</code> is missing, or a lock is taken twice by the same thread	Check that every lock has exactly one matching unlock on every path through the critical section.
<code>clock_gettime</code> or <code>CLOCK_MONOTONIC</code> undeclared	Compiled with strict <code>-std=c11</code> , which hides POSIX functions	Use <code>-std=gnu11</code> as shown in Task 4.
Task 4 timings vary widely between runs	Other applications competing for CPU, or too few increments	Close background applications, increase <code>INCREMENTS_PER_THREAD</code> , and average over all 5 runs.

11. Submission Requirements

Submit one laboratory report containing:

1. source code for `race.c` and the 10 recorded results (Task 1);
2. the ThreadSanitizer command, the captured report (`tsan_report.txt`), and the completed interpretation table (Task 2);
3. source code for `race_mutex.c`, the 10 recorded results, and evidence of a clean TSan run (Task 3);
4. source code for `counter_bench.c`, the timing table with averages and ratio, and the TSan check of the atomic version (Task 4);
5. brief answers to all discussion questions in Tasks 1 through 4;
6. representative screenshots showing successful compilation and execution of each program, including the TSan warning for Task 2.

12. Marking Rubric

Component	Marks	Assessment Focus
Task 1: Unsynchronized counter	20	Correct shared-state race implementation, 10 recorded runs with lost-update counts, and sound discussion answers
Task 2: ThreadSanitizer analysis	20	Correct TSan compilation and report capture, accurate interpretation of the reported accesses, and well-reasoned discussion answers
Task 3: Mutex fix	20	Correct critical section (lock/unlock around the update), correct results in all 10 runs, and a clean TSan re-verification

Component	Marks	Assessment Focus
Task 4: Atomic increment and comparison	30	Correct atomic implementation, fair benchmark methodology, accurate timing table, and well-reasoned comparison of correctness and performance
Code quality and submission organization	10	Readable code, meaningful identifiers, and a complete, well-organized submission
Total	100	

13. Lab Completion Checklist

Before submitting the laboratory report, verify that:

- WSL with GCC and ThreadSanitizer is available (carried over from Lab 02) and both `gcc --version` and the TSan test in Section 2.1 succeed.
- `race.c` builds and runs, and all 10 results (expected, actual, lost updates) have been recorded.
- the TSan build of `race.c` has been run, the report saved, and the interpretation table completed.
- `race_mutex.c` gives the correct total in all 10 runs, and its TSan build shows no warnings.
- `counter_bench.c` builds in all required variants with `-O2`, each variant has been run 5 times, and the averages and ratio have been recorded.
- the atomic variant(s) have been checked with TSan and show no warnings.
- all discussion questions across Tasks 1-4 are answered.
- required source code, raw output, and screenshots are included.

14. Conclusion

This laboratory examined what goes wrong when threads share mutable state without coordination, and two standard ways to repair it. Using an unsynchronized counter, students observed that concurrent increments are lost and that the final value changes from run to run, a direct consequence of `counter++` being a non-atomic load, add, and store. Using ThreadSanitizer, they then located the cause of the failure, rather than merely its symptom, and saw that a dynamic detector can flag a race even in a run whose output looks correct. Wrapping the update in a mutex-protected critical section restored correctness, and replacing the mutex with an atomic increment achieved the same correctness with a cheaper mechanism, which students measured on their own machines. Together, these observations show that correctness must come first, that tools are needed to prove it, and that the choice of synchronization mechanism is a trade-off between generality (mutexes) and cost (atomics). These concepts extend directly into later laboratories involving deadlock, synchronization primitives, and coordination between concurrent threads.